

Class Reference Sheet — Introduction to NumPy & `scipy.linalg.solve`

DA205 · Linear Algebra · Unit 1

This sheet contains everything needed to answer the Python/NumPy portions of the Unit 1 Evaluation Paper (Section A: A5, A6, A8; Section D: D1, D2), plus the supporting concepts from Section C (rank, invertibility) expressed in code. Work through it in order in a Jupyter notebook — every code block is meant to be typed and run, not just read.

1. Why NumPy?

Plain Python lists were never built for math. `numpy` gives Python a real array type — `ndarray` — that supports vectorized arithmetic, and it's the foundation every other DS/ML library sits on: `pandas`, `scikit-learn`, `PyTorch`, `TensorFlow` all build on top of NumPy arrays.

```
import numpy as np
```

This single line is assumed for every example below.

2. Creating Arrays

```
a = np.array([1, 2, 3])          # 1D array - a vector
A = np.array([[1, 2], [3, 4], [5, 6]]) # 2D array - a matrix

np.zeros((3, 3))                # 3x3 matrix of zeros
np.ones((2, 4))                 # 2x4 matrix of ones
np.eye(3)                       # 3x3 identity matrix
np.arange(0, 10, 2)             # [0, 2, 4, 6, 8] - like range(), but returns an array
np.linspace(0, 1, 5)            # 5 evenly spaced points from 0 to 1
```

Why `np.eye(3)` matters: this is the identity matrix I from $(Ax=b)$ — the matrix that does nothing under multiplication ($(AI = A)$), and shows up throughout the Invertible Matrix Theorem (Section C of the evaluation).

3. Shape, Dimensions, dtype

```
A = np.array([[1, 2], [3, 4]])

print(A.shape)    # (2, 2)  -> (rows, columns)
print(A.ndim)     # 2       -> number of dimensions
print(A.dtype)    # int64   -> data type of the entries
print(A.size)     # 4       -> total number of elements
```

This directly answers Evaluation D1. `A.shape` on a (2×2) array returns the tuple `(2, 2)` — not `(2,)` (that would be a 1D array's shape), not `2`, and not the flattened list of entries.

shape is also how you read off a "feature matrix" shape from the rank lecture: `(n_samples, n_features)`.

4. Indexing & Slicing

```
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

A[0]          # first row: [1, 2, 3]
A[0, 1]       # row 0, column 1: 2
A[:, 0]       # entire first column: [1, 4, 7]
A[1:, :]      # all rows except the first
```

Common trip-up: `A[0]` gives a **row**, not a column. Students coming from pure math notation sometimes expect column-first indexing — NumPy is always **row-major**: `A[row, column]`.

5. Reshape & Transpose

```
A.T           # transpose — flips rows and columns
np.arange(6).reshape(2, 3)  # reshape a flat 1D array into a 2x3 matrix
```

Transpose is the operation behind $(X^{\text{top}} X)$ from the normal-equations derivation — always check `.shape` before and after to build intuition for what transpose actually does to dimensions.

6. Arithmetic — Element-wise vs. Matrix Multiplication

This is the single most important distinction in this entire sheet — and it's directly tested in Evaluation A8 and D2.

```
A = np.array([[2, 3], [4, 5]])
B = np.array([[1, 0], [0, 1]])

A * B      # ELEMENT-WISE multiplication — multiplies corresponding entries
A @ B      # MATRIX multiplication — real row-by-column dot products
```

Operator	Name	What it does
----------	------	--------------

Operator	Name	What it does
$A * B$	Element-wise (Hadamard) product	$result[i,j] = A[i,j] * B[i,j]$ — requires matching shapes
$A @ B$	Matrix multiplication	Row-by-column dot products — the operation from every $(Ax=b)$ you've hand-computed

This answers Evaluation A8 and D2 directly: $A @ A$ is the matrix-multiplication answer in A8; the $*$ vs $@$ distinction is the answer to D2.

`np.dot(A, B)` is an older equivalent to $A @ B$ for 2D arrays — you may see it in older code or textbooks, but $@$ is the modern, preferred syntax (Python 3.5+).

7. Useful Aggregates

```
A.sum()           # sum of all entries
A.mean()          # mean of all entries
A.max(axis=0)     # column-wise max (axis=0 -> collapse rows)
A.max(axis=1)     # row-wise max (axis=1 -> collapse columns)
```

$axis=0$ vs $axis=1$ trips people up constantly — say it out loud as “*axis=0 squashes down the rows, axis=1 squashes across the columns*” and it tends to stick better than the formal definition.

8. `np.linalg` — Rank, Determinant, and the Invertible Matrix Theorem in Code

```
np.linalg.matrix_rank(A)  # rank(A) — same value you'd get from row-reducing by hand
np.linalg.det(A)         # determinant — nonzero iff A is invertible (for square A)
np.linalg.inv(A)         # the actual inverse matrix — avoid using this to solve Ax=b (see Section 10)
```

Direct tie to Evaluation Section C: every IMT condition you stated in words (C2) has a one-line NumPy check:

IMT condition	NumPy check
$\det(A) \neq 0$	<code>np.linalg.det(A) != 0</code>
$\text{rank}(A) = n$	<code>np.linalg.matrix_rank(A) == A.shape[0]</code>
Columns of A independent	Same as full rank, above

Worked example — the rank-2-out-of-3-columns case from your (v_1, v_2, v_3) discussion:

```
v1 = np.array([3, 1])
v2 = np.array([1, 3])
v3 = 0.7*v1 + 0.5*v2

M = np.column_stack([v1, v2, v3])
print(M)
# [[3.  1.  2.6]
#  [1.  3.  2.2]]

print(np.linalg.matrix_rank(M))  # 2 — confirms v3 adds no new direction
```

9. Importing and Using `scipy.linalg.solve` — Evaluation A5 & A6

SciPy is organized into **subpackages** — `scipy.linalg`, `scipy.optimize`, `scipy.stats`, etc. Nothing lives at the very top level of `scipy` itself except a handful of core utilities. `solve` is a function that lives specifically **inside** `scipy.linalg`.

```
# CORRECT — this is the only way that works:
from scipy.linalg import solve

# ALSO CORRECT — importing the whole subpackage, then using dot-access:
from scipy import linalg
linalg.solve(A, b)
```

```
# WRONG — will raise ModuleNotFoundError / ImportError:
import solve
```

This is exactly Evaluation A5 and A6. `import solve` fails because `solve` is not a top-level module you can import directly — it only exists as a name inside the `scipy.linalg` namespace. You must either `from scipy.linalg import solve`, or import `scipy.linalg` itself and access `linalg.solve(...)`.

10. Solving $(Ax=b)$ — Worked Example

Let's solve the exact system from your Gaussian elimination practice (2 marks in Evaluation B1, minus the row swap):

$$(x + y + z = 6, \quad 2y + z = 7, \quad 3x - 2y + 2z = 5)$$

```
from scipy.linalg import solve
import numpy as np

A = np.array([[1, 1, 1],
              [0, 2, 1],
              [3, -2, 2]])
b = np.array([6, 7, 5])

x = solve(A, b)
print(x)                # [1. 2. 3.]

print(np.allclose(A @ x, b)) # True -> verifies the solution
```

Why `scipy.linalg.solve` and not `np.linalg.inv(A) @ b`?

Both give the same answer here, but `solve()` is preferred because:

- It's **faster** — computing a full inverse does more work than necessary
- It's **more numerically stable** — errors compound less
- Internally, it performs **LU decomposition** — literally the row operations from Gaussian elimination, automated and optimized

```
# Avoid this pattern:
x_bad = np.linalg.inv(A) @ b # works, but slower and less stable

# Prefer this:
x_good = solve(A, b)
```

11. What Happens When (A) Isn't Invertible

```
A_bad = np.array([[1, 2], [2, 4]]) # second row is 2x the first -> rank 1, not 2
b_bad = np.array([3, 6])

print(np.linalg.matrix_rank(A_bad)) # 1 -- not full rank
print(np.linalg.det(A_bad)) # 0.0

x = solve(A_bad, b_bad) # raises LinAlgError, or a LinAlgWarning + garbage numbers
```

Best practice — check rank before trusting the answer:

```
def safe_solve(A, b, tol=1e-8):
    n = A.shape[0]
    if A.shape[0] != A.shape[1]:
        raise ValueError(f"A must be square, got {A.shape}")
    if np.linalg.matrix_rank(A, tol=tol) < n:
        raise ValueError("A is rank-deficient — no unique solution exists.")
    return solve(A, b)
```

This is the Invertible Matrix Theorem, enforced as code, before you ever get a misleading number back.

Evaluation Paper — Cross-Reference Map

Evaluation question	Covered in section
A5 (correct <code>numpy.linalg.solve</code> import)	Section 9
A6 (why plain <code>import solve</code> fails)	Section 9
A8 (<code>A @ A</code> = matrix multiplication)	Section 6
D1 (<code>A.shape</code> output)	Section 3
D2 (<code>A * B</code> vs <code>A @ B</code>)	Section 6
C1/C2 (rank, IMT conditions)	Section 8, 10, 11

References & Further Reading

Official documentation (primary source — always check these first):

- NumPy Quickstart Tutorial — numpy.org/doc/stable/user/quickstart.html (<https://numpy.org/doc/stable/user/quickstart.html>)
- NumPy: The Absolute Basics for Beginners — numpy.org/doc/stable/user/absolute_beginners.html (https://numpy.org/doc/stable/user/absolute_beginners.html)
- `numpy.linalg.solve` reference — docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.solve.html (<https://docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.solve.html>)

- `numpy.linalg` module overview — numpy.org/doc/stable/reference/routines.linalg.html (<https://numpy.org/doc/stable/reference/routines.linalg.html>).

Books:

- **Gilbert Strang**, *Linear Algebra and Its Applications* (5th Edition), Cengage Learning — already on your course reading list; Chapter 2 (elimination) and Chapter 2.5 (inverse matrices) map directly to Sections 8–11 here.
- **Wes McKinney**, *Python for Data Analysis* (3rd Edition), O'Reilly — Chapter 4 is a thorough, practical NumPy array reference beyond what's covered in this sheet.
- **Lloyd N. Trefethen & David Bau III**, *Numerical Linear Algebra*, SIAM — for students who want the theory behind *why* LU decomposition (what `scipy.linalg.solve` does internally) is numerically preferred over computing a matrix inverse directly.

Free supplementary tutorials:

- *NumPy Illustrated: The Visual Guide to NumPy* by Lev Maximov — a widely used visual/intuitive companion, linked from NumPy's own official "Learn" page.
- Stanford CS231n NumPy tutorial — a concise, practical NumPy primer originally written for a machine learning course, also linked from NumPy's official resources page.

